

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

PROJECT REPORT

Team ID: SWTID-2026-2091

Date: 30 September 2026

GOVERNMENT ARTS AND SCIENCE COLLEGE, KADAYANALLUR
BSC.COMPUTER SCIENCE
2024-2027

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Overview

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement

2.2 Empathy Map Canvas

2.3 Brainstorming

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

3.2 Solution Requirement

3.3 Data Flow Diagram

3.4 Technology Stack

4. PROJECT DESIGN

4.1 Problem Solution Fit

4.2 Proposed Solution

4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

7. RESULTS

7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

Source Code (if any)

Dataset Link

GitHub & Project Demo Link

1. INTRODUCTION

1.1 Project Overview

The project titled “Implement Client Script & UI Policy (Incident)” focuses on improving the behaviour of the Incident form in ServiceNow. Client Script and UI Policy are used to control and validate form behaviour on the client side. The project demonstrates how these configurations can make Incident form interactions more consistent, controlled, and user-friendly.

The Client Script is used to perform client-side actions or validation when the user interacts with the Incident form. The UI Policy is used to dynamically control field behaviour based on specified conditions. Together, they help reduce incorrect data entry and provide a better Incident management experience.

1.2 Purpose

The purpose of this project is to implement and test Client Script and UI Policy functionality on the ServiceNow Incident table. The project aims to ensure that the required form behaviour is applied automatically according to defined conditions and that users receive a consistent experience while creating or updating Incident records.

2. IDEATION PHASE

2.1 Problem Statement

In an Incident management system, users may enter incomplete or inappropriate information if form fields do not respond according to the situation. Manual checking can also lead to inconsistent data entry. The project addresses this problem by using ServiceNow Client Script and UI Policy to provide automatic client-side validation and dynamic field behaviour on the Incident form.

2.2 Empathy Map Canvas

Category	Project Understanding
Says	The user wants the Incident form to be simple and clear to use.
Thinks	The user expects the form to guide them and prevent incorrect entries.
Does	The user creates, updates, and submits Incident records.
Feels	The user may become confused when required information or field behaviour is not clear.

2.3 Brainstorming

- Identify the fields and conditions that require dynamic behaviour.
- Use Client Script for client-side validation or automatic actions.
- Use UI Policy to control field visibility, mandatory status, or read-only behaviour.
- Test the configuration using different Incident form conditions.
- Verify that the implemented behaviour works consistently.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

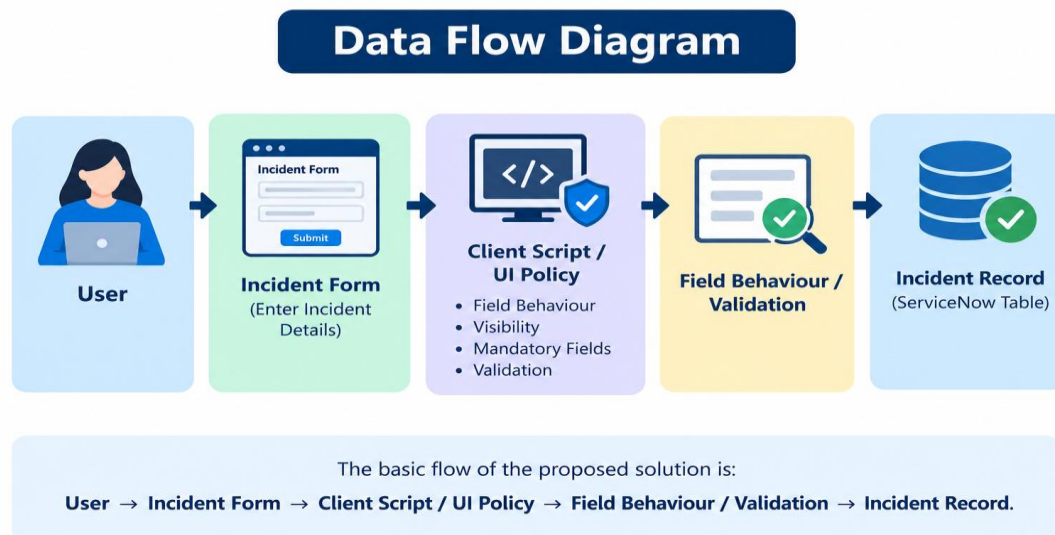
Stage	User Activity	System Behaviour
Open	User opens the Incident form.	ServiceNow displays the Incident form.
Enter	User enters Incident information.	Client-side rules respond to user input where configured.
Condition	Configured condition becomes true/false.	UI Policy changes the configured field behaviour.
Validate	User checks the form.	Client Script performs configured client-side action/validation.
Submit	User submits the Incident.	Incident record is created or updated according to the form rules.

3.2 Solution Requirement

- ServiceNow Incident table and form.
- Configured Client Script for the required client-side behaviour.
- Configured UI Policy with suitable conditions and UI Policy Actions.
- Test Incident records to validate the implemented functionality.
- Access to a ServiceNow instance for configuration and testing.

3.3 Data Flow Diagram

The basic flow of the proposed solution is: User → Incident Form → Client Script / UI Policy → Field Behaviour / Validation → Incident Record.



3.4 Technology Stack

Technology / Component	Purpose
ServiceNow	Platform used to implement and test the project.
Incident Table	Target table on which the configuration is applied.
Client Script	Used for client-side actions and validation.
UI Policy	Used to control field behaviour dynamically.
JavaScript	Used where scripting is required in the Client Script.
Web Browser	Used to access and test the ServiceNow application.

4. PROJECT DESIGN

4.1 Problem Solution Fit

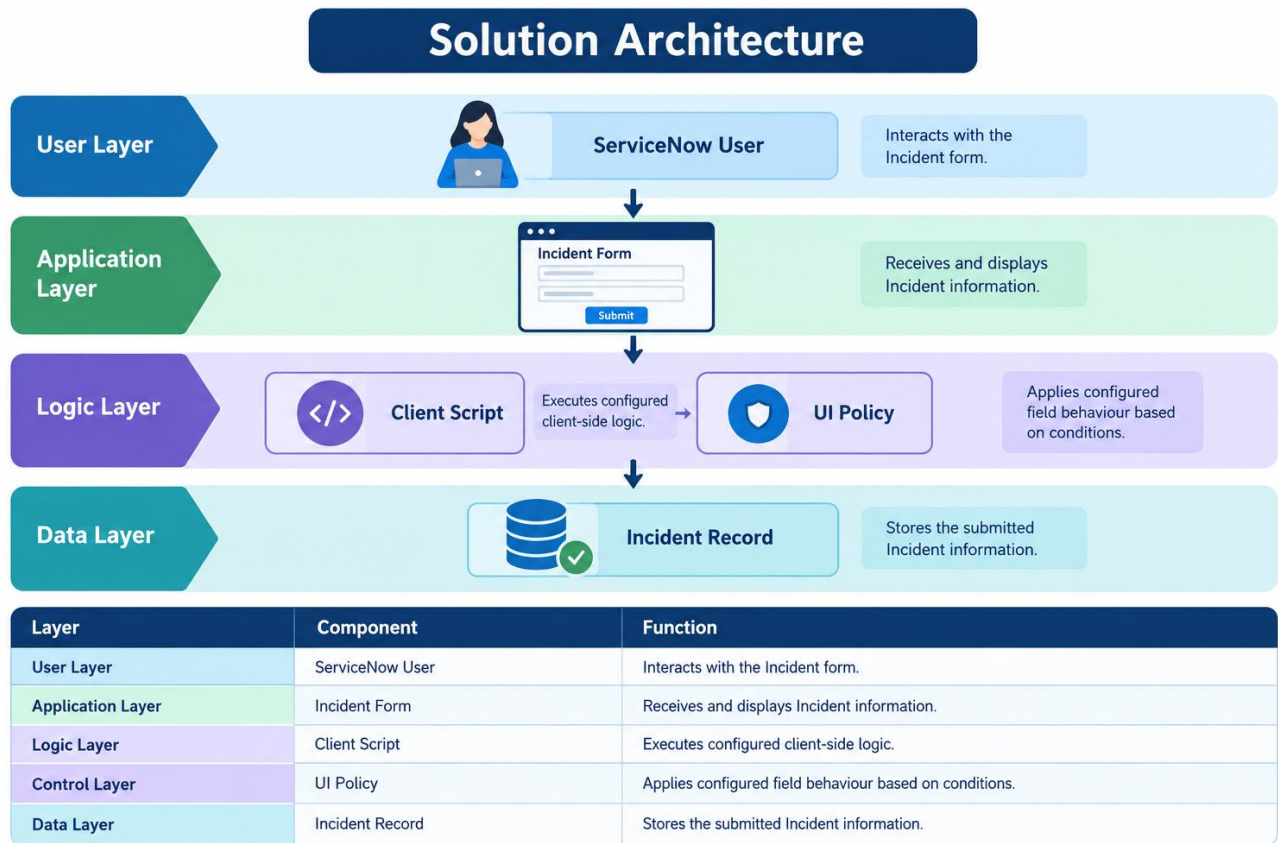
The identified problem is inconsistent or insufficient form guidance during Incident creation and update. The solution fits the problem by applying client-side rules directly on the Incident form. Client Script handles configured client-side logic, while UI Policy manages field behaviour based on defined conditions.

4.2 Proposed Solution

The proposed solution is to configure a Client Script and a UI Policy for the Incident form. The Client Script performs the required client-side operation, while the UI Policy and its actions dynamically control the selected field behaviour. The configuration is then tested using Incident records to confirm the expected result.

4.3 Solution Architecture

Layer	Component	Function
User Layer	ServiceNow User	Interacts with the Incident form.
Application Layer	Incident Form	Receives and displays Incident information.
Logic Layer	Client Script	Executes configured client-side logic.
Control Layer	UI Policy	Applies configured field behaviour based on conditions.
Data Layer	Incident Record	Stores the submitted Incident information.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Activity	Status
Ideation	Identify problem and generate solution ideas	Completed
Requirement Analysis	Identify requirements, journey and data flow	Completed
Design	Prepare solution design and architecture	Completed
Implementation	Configure Client Script and UI Policy	Completed
Testing	Perform functional and performance testing	Completed
Results	Capture output screenshots and prepare report	Completed

The project was developed phase-by-phase, starting from problem identification and requirement analysis, followed by design, implementation, testing, and documentation.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional testing was performed to verify that the Client Script and UI Policy behave according to the configured conditions on the Incident form. Performance testing focused on form responsiveness and stability while using the configured functionality.

Test Case ID	Scenario	Test Steps	Expected Result	Actual Result	Result
FT-01	Client Script Validation	Open Incident form and test the configured Client Script condition.	Configured client-side behaviour should occur.	As expected*	Pass*
FT-02	UI Policy Validation	Open Incident form and test the configured UI Policy condition.	Configured field behaviour should change as defined.	As expected*	Pass*
FT-03	Incident Form Behaviour	Create/open an Incident and provide required values.	Client Script and UI Policy should work correctly.	As expected*	Pass*
FT-04	Functional Integration Check	Test Client Script and UI Policy together.	Both configurations should work without conflict.	As expected*	Pass*
PT-01	Incident Form Response Test	Open form and trigger the configured rules.	Form should respond without noticeable delay.	As expected*	Pass*
PT-02	Multiple Incident Test	Test the configuration on multiple Incident records.	Configuration should work consistently.	As expected*	Pass*
PT-03	Form Stability Test	Create/update Incident records while triggering rules.	Form should work smoothly without errors.	As expected*	Pass*

*Replace the Actual Result and Pass/Fail values with the observed result if your testing produced a different outcome.

7. RESULTS

7.1 Output Screenshots

The following screenshots should be inserted as evidence of the implemented project.

Figure 7.1: Client Script configuration

Figure 7.2: UI Policy configuration

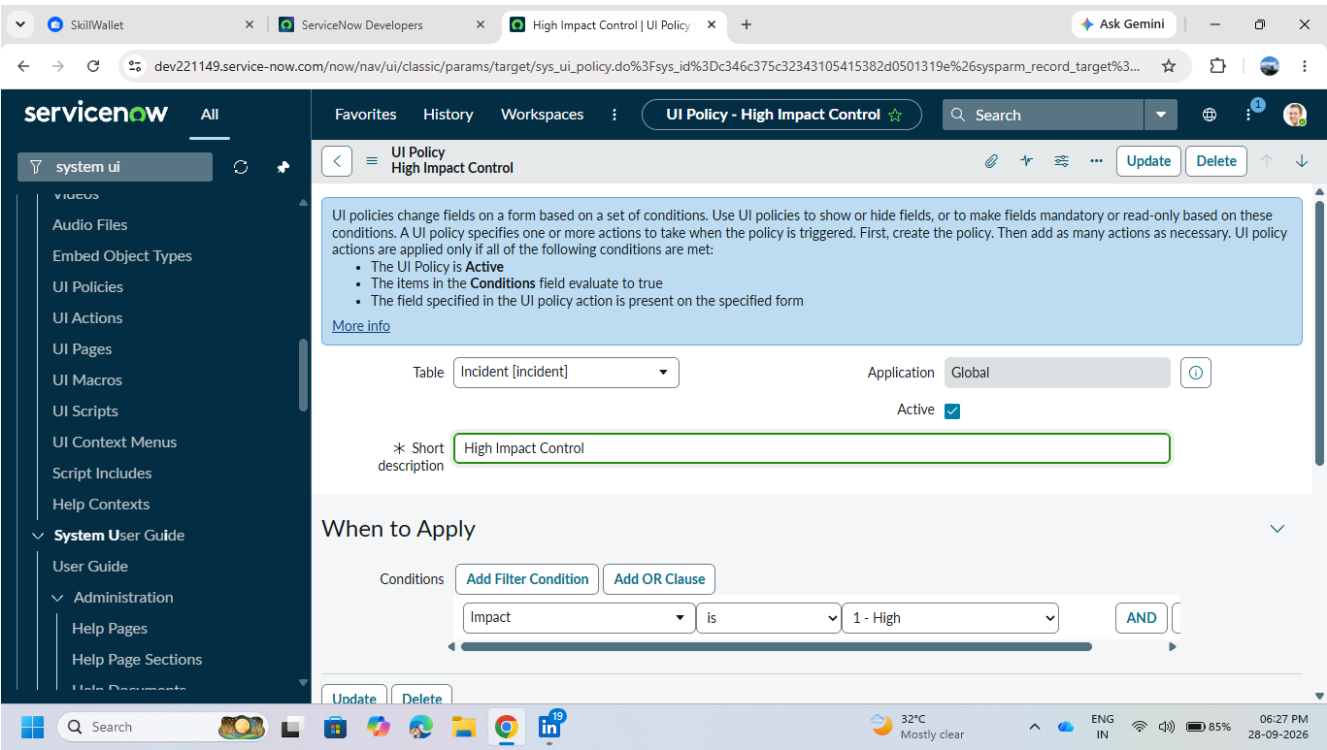


Figure 7.3: UI Policy condition / UI Policy Action

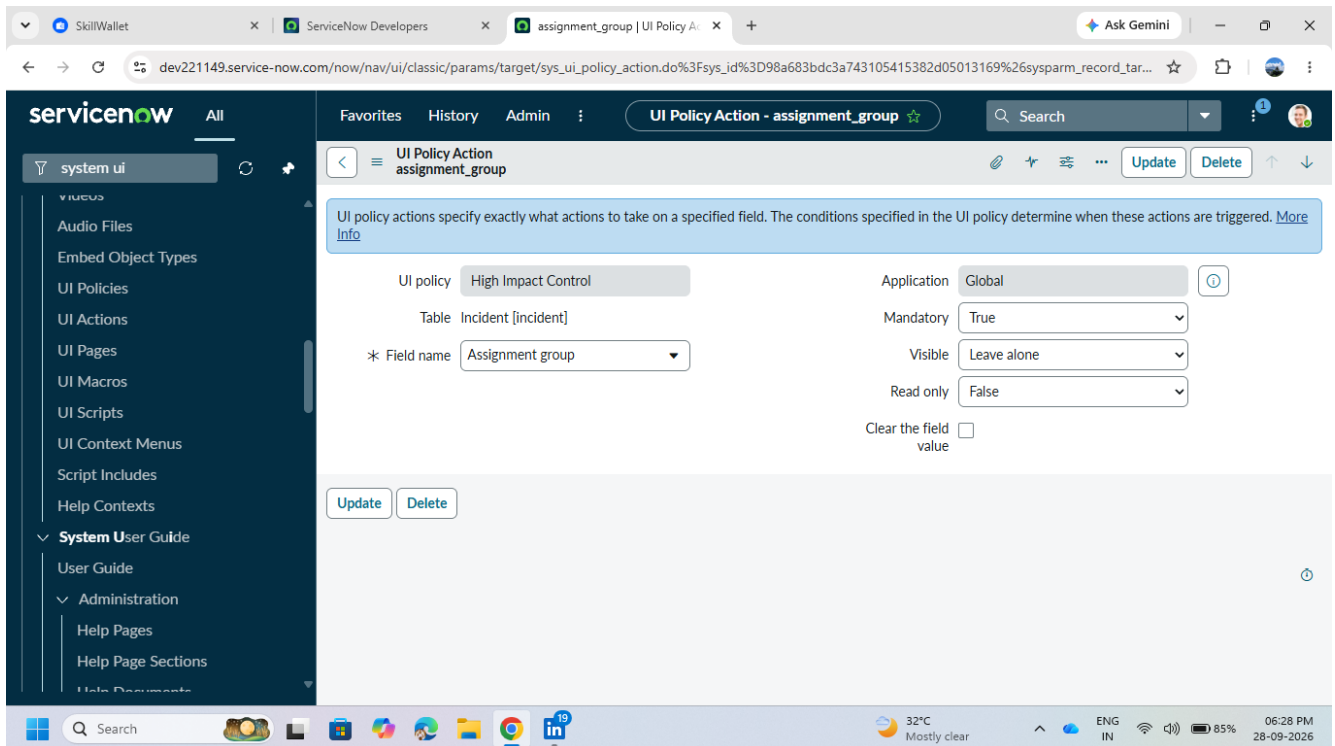


Figure 7.4: Incident form before applying the configured condition

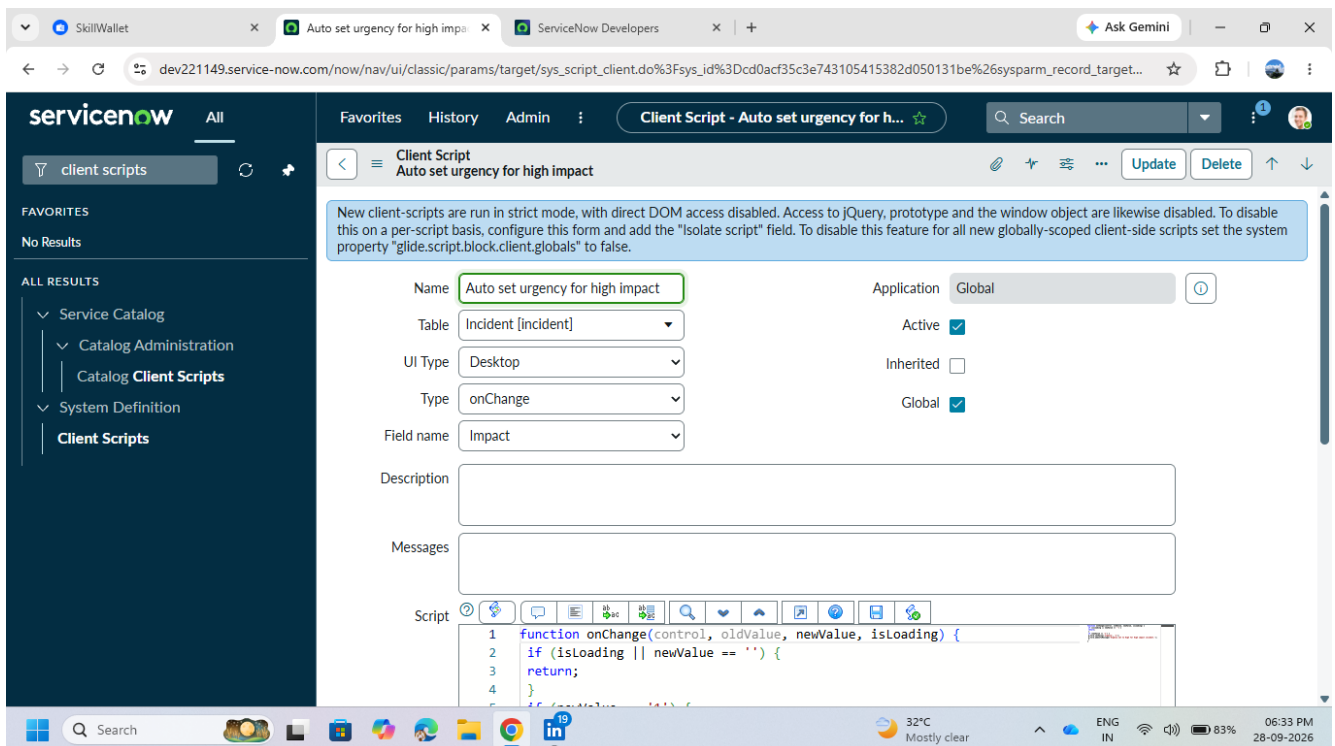


Figure 7.5: Incident form showing the Client Script / UI Policy behaviour

dev221149.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3D-1%26sysparm_query%3Dactive%3Dtrue%26sysparm_stack%3Dincident_lis...

Incident - Create INC0010007

Urgency set to High for High impact incident.

Number: INC0010007

Channel: -- None --

* Caller: Abraham Lincoln

State: New

Category: Inquiry / Help

Impact: 1 - High

Subcategory: -- None --

Urgency: 1 - High

Service:

Priority: 1 - Critical

Service offering:

* Assignment group: CAB Approval

Configuration item:

Assigned to:

* Short description: cab approve

Description:

Assigned To is mandatory for High impact incidents.

Figure 7.6: Final testing result

dev221149.service-now.com/now/nav/ui/classic/params/target/incident_list.do%3Factive%3Dtrue%26sysparm_query%3Dactive%253Dtrue%255EEQ

Incidents

All > Active = true

	Number	Opened	Short description	Caller	Priority	State	Category	Assignment group
☐	INC0010007	2026-09-28 05:58:04	cab approve	Abraham Lincoln	2 - High	New	Inquiry / Help	CAB Approval
☐	INC0010002	2026-09-26 07:23:21	practise	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval
☐	INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)
☐	INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)
☐	INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)
☐	INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)
☐	INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)
☐	INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)

1 to 20 of 42

8. ADVANTAGES & DISADVANTAGES

Advantages

- Improves consistency in Incident form behaviour.
- Reduces incorrect or incomplete data entry through configured client-side rules.
- Provides dynamic field behaviour based on conditions.
- Improves usability by guiding users during Incident creation and update.
- Can be tested and maintained within the ServiceNow platform.

Disadvantages

- Client-side rules mainly affect the user interface and do not replace server-side data validation.
- Incorrect configuration can cause unexpected form behaviour.
- The solution depends on the ServiceNow platform and its configuration environment.

9. CONCLUSION

The project successfully demonstrates the implementation of Client Script and UI Policy on the ServiceNow Incident form. The Client Script provides configured client-side logic, while the UI Policy controls field behaviour according to defined conditions. Functional and performance testing can be used to confirm that the configuration works consistently. Overall, the project provides a practical example of improving Incident form behaviour through ServiceNow configuration.

10. FUTURE SCOPE

- Additional Client Scripts and UI Policies can be introduced for more Incident scenarios.
- More validation rules can be added to improve data quality.
- Server-side validation can be combined with client-side rules for stronger data control.
- Additional automation can be integrated with Incident workflows.
- User feedback and testing results can be used to refine the form experience.

11. APPENDIX

Source Code (if any)

The Client Script source code can be included below or attached as supporting project material.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback)
{
    alert('State cannot be updated using list editing. Please open the Incident.');
```



```
    callback(false);
}
```

Dataset Link

Not Applicable – This ServiceNow configuration project does not use an AI/ML dataset.

GitHub & Project Demo Link

GitHub Repository: <https://github.com/rmahalakshmi411/Implement-Client-Script-UI-Policy-Incident->

Project Demo Video: <https://drive.google.com/file/d/1tZZa1w-Avzj4ZBC3JwCxMma8xSsr9bZA/view?usp=drivesdk>

SUBMISSION CHECKLIST

- Replace [COLLEGE NAME], [DEPARTMENT NAME], and [ACADEMIC YEAR].
- Insert the Data Flow Diagram.
- Insert the Solution Architecture diagram.
- Insert actual ServiceNow screenshots in Chapter 7.
- Paste the final Client Script code in Appendix.
- Paste the GitHub repository link.
- Paste the Project Demo video link.
- Verify the actual test results before final submission.
- Add page numbers and update the Table of Contents in Microsoft Word if required.